

1) What is Java?

Java is a high-level, class-based, **object-oriented programming language** designed to have as few implementation dependencies as possible. Launched by Sun Microsystems in 1995 (now owned by Oracle), it operates on the principle of "**Write Once, Run Anywhere**" (WORA).

Key Features

- **Object-Oriented:** Everything in Java is an "Object," which makes the code modular and reusable.
- **Platform Independent:** Java code doesn't rely on the underlying hardware.
- **Robust:** It emphasizes compile-time and runtime error checking.
- **Secure:** Features like the bytecode verifier and lack of explicit pointers make it highly secure.
- **Multithreaded:** Java can perform multiple tasks simultaneously within a single program.

2) What is Platform Independence?

Platform independence means that a program written on one type of operating system (like Windows) can execute on another (like macOS or Linux) without any modifications.

In Java, this is achieved because the compiler does not produce an executable file (like an `.exe`). Instead, it produces **Bytecode**, which can be run on any device equipped with a **Java Virtual Machine (JVM)**.

3) Difference between JDK, JRE, and JVM

To understand these, think of them as layers:

Component	Full Name	Purpose
JVM	Java Virtual Machine	The "engine" that actually runs the bytecode. It is platform-dependent.
JRE	Java Runtime Environment	Contains the JVM + Library files needed to run Java programs.
JDK	Java Development Kit	Contains the JRE + Development tools (compiler, debugger) to create Java programs.

4) How does JVM work?

The JVM performs three main functions:

Loads code: Using the Class Loader.

Verifies code: Ensuring the bytecode doesn't violate security constraints.

Executes code: Using an Execution Engine (which includes a JIT compiler to turn bytecode into native machine code).

5) What is Bytecode?

Bytecode is the intermediate representation of your Java code. It is a highly optimized set of instructions that is not readable by humans or directly by the computer hardware. It is only "understandable" by the JVM.

6) What is Java Compilation Process

The journey from text to a running program looks like this:

Source Code: You write code in a `.java` file.

Compilation: The Java Compiler (`javac`) converts the source code into **Bytecode**.

Output: A `.class` file is generated.

Execution: The JVM reads the `.class` file and translates it into machine code for the specific OS.

7) Java Editions

Java SE (Standard Edition): The core Java programming platform. It includes APIs for basic functionality (strings, networking, GUI).

Java EE (Enterprise Edition): Built on top of SE, used for developing large-scale, distributed enterprise applications (web servers, etc.).

Java ME (Micro Edition): A subset of SE designed for small devices like embedded systems and mobile phones.

8) What is Classpath?

The **Classpath** is an environment variable that tells the Java compiler and the JVM where to look for user-defined classes and packages. If your code relies on external libraries, you must include their location in the Classpath so Java can find them.

9) How to Set Up the Java Environment

Download JDK: Get the latest version from the official Oracle website or an open-source distribution like OpenJDK.

Install: Run the installer.

Set Environment Variables: * Set `JAVA_HOME` to the directory where the JDK is installed.

- a. Add the `bin` folder of the JDK to your system's `PATH` variable.

Verify: Open a terminal and type `java -version`.

10) Compiled vs. Interpreted Languages

Java is unique because it is **both** compiled and interpreted.

Compiled Languages (e.g., C++): The entire source code is converted into machine code at once before execution. It's very fast but platform-specific.

Interpreted Languages (e.g., Python): The code is read and executed line-by-line. It's flexible but generally slower.

Java's Hybrid Approach: Java **compiles** source code into bytecode (for portability) and then the JVM **interprets** (or JIT compiles) that bytecode into machine code (for execution).

11) Data Types in Java

Data types specify the size and type of values that can be stored in an identifier. Java is **statically-typed**, meaning all variables must be declared before they can be used.

12) Primitive vs. Non-Primitive Types

Feature	Primitive Types	Non-Primitive (Reference) Types
Definition	Predefined by Java (e.g., int, char).	Defined by the programmer (except String).
Value	Stores the actual value.	Stores a memory address (reference) to the object.
Nullability	Cannot be null.	Can be null.
Size	Depends on the specific type (fixed).	All references have the same size.
Examples	byte, short, int, long, float, double, boolean, char.	String, Arrays, Classes, Interfaces.

13)What are Variables and Their Types

A variable is a container that holds data during the execution of a program.

Local Variables: Declared inside a method; visible only within that method.

Instance Variables: Declared inside a class but outside methods; unique to each "instance" (object).

Static (Class) Variables: Declared with the `static` keyword; shared across all instances of a class.

14)What is Type Casting

Type casting is when you assign a value of one primitive data type to another type.

Implicit (Widening): Happens automatically when passing a smaller type to a larger type.

a. *Example:* int to double.

Explicit (Narrowing): Must be done manually by placing the type in parentheses. Potential for data loss.

b. *Example:* double myDouble = 9.78; int myInt = (int) myDouble;
(Result: 9)

15) Operators in Java

Arithmetic: +, -, *, /, %

Relational: ==, !=, >, <, >=, <=

Logical: && (AND), || (OR), ! (NOT)

Assignment: =, +=, -=, *=, /=

Unary: ++, --

16) Difference between == and equals()

== (Operator): Compares **memory addresses** (references). It checks if both objects point to the exact same location in memory. For primitives, it compares actual values.

equals() (Method): Compares the **content** or state of the objects.

Note: If you compare two different String objects with the same text using ==, it might return false, but .equals() will return true.

17) what are Control Statements

These determine the flow of program execution.

Selection Statements

- **if / else:** Executes a block of code if a condition is true.
- **switch:** Selects one of many code blocks to be executed based on a variable's value (useful for replacing multiple if-else chains).

Iteration Statements (Loops)

- **for:** Used when you know exactly how many times you want to loop.
- **while:** Checks the condition **before** executing the block.
- **do-while:** Executes the block **once** before checking the condition (guarantees at least one execution).

Break and Continue

- **break:** Immediately terminates the loop or switch statement and moves to the next line of code after the block.
- **continue:** Skips the current iteration of the loop and jumps to the next evaluation of the loop's condition.

18)What is Ternary Operator

It is a shorthand for an if-else statement. It uses three operands. **Syntax:** `variable = (condition) ? valueIfTrue : valueIfFalse;` *Example:* `String result = (marks > 50) ? "Pass" : "Fail";`

19)What is Scope of a Variable

Scope refers to the region of the program where a variable is accessible.

- **Class Level:** Accessible everywhere in the class (Instance/Static variables).
- **Method Level:** Accessible only within the method where it was declared.
- **Block Level:** If a variable is declared inside a loop or an if block, it "dies" once that block finishes.

20) What is OOP?

Object-Oriented Programming is a programming paradigm based on the concept of "objects," which can contain data (fields) and code (methods). It aims to implement real-world entities like inheritance, hiding, and polymorphism in programming to make code more reusable, scalable, and manageable.

21) what are OOP Principles

The four pillars of OOP are:

1. **Encapsulation:** Wrapping data and methods into a single unit (class) and restricting direct access.
2. **Inheritance:** Allowing one class to acquire the properties and behaviors of another.
3. **Polymorphism:** The ability of a single action to perform in different ways (e.g., one method name with different behaviors).
4. **Abstraction:** Hiding complex implementation details and showing only the essential features of an object.

22) what are Classes & Objects

- **Class:** A **blueprint** or template for creating objects. It doesn't occupy memory until an object is created.
- **Object:** An **instance** of a class. It is a physical entity that occupies memory.

Feature	Class	Object
Nature	Logical template.	Physical reality.
Memory	No memory allocated on declaration.	Memory allocated when created using new.
Count	Declared once.	Many objects can be created from one class.

23) Constructors

A constructor is a special block of code used to **initialize objects**. It is called automatically when an object is created.

- **Rules:** It must have the same name as the class and cannot have a return type (not even void).

Types of Constructors

1. **Default Constructor:** Provided by the Java compiler if you don't write one. It initializes fields with default values (e.g., 0 for int).
2. **No-Arg Constructor:** A constructor written by the programmer that accepts no parameters.
3. **Parameterized Constructor:** A constructor that accepts arguments to initialize fields with specific values.

24) What is Encapsulation / Getters and setters

Encapsulation

Encapsulation is the process of bundling data (variables) and methods (code) together.

- How to achieve it: Declare variables as private and provide public Getter and Setter methods.
- Why use Getters/Setters? They provide control. You can add validation logic inside a setter (e.g., if (age > 0)) to ensure the object's data remains valid.

25) Inheritance

Inheritance allows a **Subclass** (child) to inherit fields and methods from a **Superclass** (parent).

Types of Inheritance in Java

1. **Single:** Class B extends Class A.
2. **Multilevel:** Class C extends Class B, which extends Class A.
3. **Hierarchical:** Class B and Class C both extend Class A.

Note: Multiple Inheritance (one class extending two classes) is **not supported** in Java using classes to avoid the **Diamond Problem** (ambiguity if both parents have a method with the same name).

26) Polymorphism

Method Overloading vs. Overriding

Feature	Overloading (Compile-time)	Overriding (Runtime)
Location	Within the same class.	Between Superclass and Subclass.
Parameters	Must be different.	Must be exactly the same.
Binding	Static binding.	Dynamic binding.

What is Runtime Polymorphism?

It is a process in which a call to an overridden method is resolved at runtime rather than compile-time. This is achieved through **Method Overriding** and **Upcasting** (e.g., Parent obj = new Child();).

27) Abstraction

Abstract Class vs. Interface

Feature	Abstract Class	Interface
Methods	Can have both abstract and concrete methods.	Mostly abstract (until Java 8, which added default/static methods).
Variables	Can have final, non-final, static, and non-static.	Variables are implicitly public static final.

Feature	Abstract Class	Interface
Inheritance	A class can extend only one abstract class.	A class can implement multiple interfaces.

When to use an Interface?

Use an interface when you want to define a **contract** for what a class should do, without caring about how it does it. It is the best tool for achieving **total abstraction** and **multiple inheritance** in Java.

28) Exception Handling

An **Exception** is an unwanted or unexpected event that occurs during the execution of a program (at runtime) which disrupts the normal flow of instructions. Handling these allows the program to continue running instead of crashing.

Types of Exceptions

Java exceptions are organized into a hierarchy, primarily divided into:

- **Checked Exceptions:** These are checked at **compile-time**. The compiler forces you to either handle them (try-catch) or declare them (throws).
 - *Examples:* IOException, SQLException, ClassNotFoundException.
- **Unchecked Exceptions:** These occur at **runtime**. The compiler does not check for them. They usually result from logic errors.
 - *Examples:* ArithmeticException (dividing by zero), NullPointerException, ArrayIndexOutOfBoundsException.

29) Difference between Error and Exception

Both are subclasses of the Throwable class, but they serve different purposes:

Feature	Exception	Error
Nature	Caused by the application logic.	Caused by environment/system resources.
Recoverability	Can be caught and handled.	Generally irrecoverable.

Feature	Exception	Error
Examples	FileNotFoundException, NullPointerException.	OutOfMemoryError, StackOverflowError.

30) Try, Catch, and Finally

These keywords form the backbone of error handling:

- **try block:** Contains the code that *might* throw an exception.
- **catch block:** Contains the code to handle the specific exception if it occurs. You **can have multiple catch blocks** for a single try block to handle different types of exceptions separately.

Rule: Always put specific exceptions (like `ArithmeticException`) before the general Exception class in multiple catch blocks.

- **finally block:** A block that **always executes**, whether an exception was handled or not. It is typically used for "cleanup" activities like closing file streams or database connections.

31) Throw vs. Throws

Though they sound similar, they serve very different roles:

- **throw:** Used to **explicitly throw** an exception from a method or block of code.
 - *Example:* `throw new ArithmeticException("Don't divide by zero!");`
- **throws:** A keyword used in a **method signature** to declare that this method might throw certain exceptions. It "passes the buck" to the caller of the method to handle the exception.

32) Custom Exception

A custom exception (or User-defined exception) is created by extending the Exception class (for checked) or RuntimeException class (for unchecked).

- **Why?** To represent specific business errors, like `InadequateFundsException` in a banking app.
-

Exception Propagation

When an exception occurs, it is first thrown from the top of the "stack" (the current method). If it isn't caught there, it "drops down" to the previous method that called it. This continues until the exception is either caught or it reaches the main method and terminates the program.

- **Note:** By default, **unchecked exceptions** are automatically propagated, whereas **checked exceptions** are not propagated unless you use the throws keyword.

33) Can we have multiple catch blocks?

Yes. A single try block can be followed by multiple catch blocks. This allows you to handle different errors in different ways.

Java

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero!");  
} catch (Exception e) {  
    System.out.println("A general error occurred.");  
}
```

34) What is Java Collections Framework

The **Collections Framework** is a unified architecture for storing and manipulating a group of objects. It provides a set of interfaces (like List, Set, Queue) and classes (like ArrayList, HashMap) that reduce programming effort by providing high-performance data structures.

35) Difference between List, Set, and Map

These are the three main interfaces of the framework:

Feature	List	Set	Map
Duplicates	Allowed.	Not allowed.	Duplicate keys not allowed; values can be.

Feature	List	Set	Map
Order	Maintains insertion order.	Generally unordered (except LinkedHashSet/TreeSet).	Key-value pairs; order depends on implementation.
Indexing	Uses index (0, 1, 2...).	No index-based access.	Uses Keys to access Values.

36) List: ArrayList vs. LinkedList

- ArrayList: Uses a dynamic array to store elements.
 - Best for: Frequent data retrieval (Search).
 - Worst for: Frequent insertion or deletion in the middle, as it requires shifting elements.
- LinkedList: Uses a doubly linked list (each element stores a pointer to the next and previous).
 - Best for: Frequent insertion and deletion.
 - Worst for: Search, as it must traverse the list from the start/end.

Why is ArrayList faster than LinkedList?

ArrayList is faster for accessing elements ($O(1)$ time complexity) because it uses an index. Since elements are stored in contiguous memory locations, the CPU can cache the data more effectively. LinkedList requires $O(n)$ time to find an element by index.

Set: HashSet vs. TreeSet

A Set is a collection that contains no duplicate elements.

- HashSet: Uses a hash table. It is unordered and allows one null value. It is very fast for basic operations ($O(1)$).
 - TreeSet: Uses a Tree structure (Red-Black tree). It stores elements in sorted (ascending) order. It is slower than HashSet ($O(\log n)$).
-

Map: HashMap vs. Hashtable

Both store Key-Value pairs, but they have critical differences:

Feature	HashMap	Hashtable
Thread Safety	Not synchronized (Not thread-safe).	Synchronized (Thread-safe).
Null Values	Allows one null key and multiple null values.	Does not allow null keys or values.
Performance	Fast.	Slow (due to locking overhead).

- **LinkedHashMap:** A variation of HashMap that maintains the insertion order. It uses a doubly-linked list running through all of its entries.

37) What is an Iterator?

An **Iterator** is an object used to loop through collections (like List and Set). It allows you to traverse the collection and safely **remove** elements during iteration, which a standard for-each loop cannot do without throwing a `ConcurrentModificationException`.

Comparable vs. Comparator

- **Comparable:** Used for **Natural Sorting** (e.g., numbers in ascending order). The class itself must implement `Comparable` and override `compareTo()`.
- **Comparator:** Used for **Custom Sorting** (e.g., sorting Employees by Salary). It is a separate class/lambda that implements `compare()`, allowing you to define multiple different sorting logic for the same object.

38) What is sorting in collections?

Sorting is the process of arranging elements in a specific order.

- To sort a **List**, use `Collections.sort(list)`.
- To sort a **Map** by keys, use `TreeMap`.
- To sort a **Set**, use `TreeSet`.

39) What is Stream API?

Introduced in Java 8, the **Stream API** is used to process collections of objects in a functional way. It allows for declarative transformations (like filter, map, reduce) and can leverage multi-core processors easily via `parallelStream`.

- *Example:* `list.stream().filter(x -> x > 10).collect(Collectors.toList());`

40) What is a Lambda Expression?

A **Lambda Expression** is an anonymous function (a method without a name) that is used to provide the implementation of a method defined in a functional interface. It enables you to treat functionality as a method argument, or code as data.

Syntax of Lambda:

`$(parameters) -> { body }$`

- **No parameter:** `() -> System.out.println("Hello");`
- **One parameter:** `(name) -> System.out.println(name);`
- **Multiple parameters:** `(a, b) -> a + b;`

41) What is a Functional Interface?

A **Functional Interface** is an interface that contains **exactly one abstract method**. It can have multiple default or static methods, but only one abstract one.

- They are annotated with `@FunctionalInterface` (optional, but recommended for compiler checking).

Examples of Built-in Functional Interfaces:

1. **Predicate:** Takes one argument and returns a boolean (used for filtering).
2. **Function<T, R>:** Takes one argument and returns a result.
3. **Consumer:** Takes an argument and returns nothing (void).
4. **Supplier:** Takes no arguments and returns a value.

42) What is Method Reference?

Method reference is a shorthand notation for a lambda expression that simply calls an existing method. It uses the double colon `::` operator.

- **Lambda:** `str -> System.out.println(str)`, **Method Reference:** `System.out::println`

43) What is the Optional Class?

Optional<T> is a container object used to represent the presence or absence of a value.

- **Purpose:** To prevent the dreaded NullPointerException (NPE).
- Instead of returning null, a method returns an Optional. You can then use methods like .isPresent(), .orElse(), or .ifPresent() to handle the result safely.

44) What is Stream API?

The **Stream API** is a powerful tool for processing sequences of elements (like Collections) in a functional style. A "Stream" is not a data structure; it conveys elements from a source (like a List) through a pipeline of computational steps.

Intermediate vs. Terminal Operations

The Stream pipeline consists of two types of operations:

Feature	Intermediate Operations	Terminal Operations
Function	Transforms a stream into another stream.	Produces a result or side-effect.
Laziness	They are lazy ; they don't execute until a terminal operation is called.	Triggers the actual processing of the stream.
Examples	filter(), map(), sorted(), distinct().	collect(), forEach(), count(), reduce().

45) What is a Parallel Stream?

A **Parallel Stream** is a stream that splits the data into multiple chunks, processing each chunk on a different CPU core simultaneously using the **ForkJoinPool**.

- **When to use:** On very large datasets where the operations are computationally expensive.
- **Caveat:** It can actually be *slower* for small datasets due to the overhead of managing multiple threads, and it should not be used if the order of processing matters or if there are shared state issues. **Java Example:**

```
long count = list.parallelStream()
                .filter(x -> x > 100).count();
```

46) Java I/O and File Handling

I/O (Input/Output) in Java is used to process data from a source (like a file or keyboard) to a destination (like a file or console).

What is an I/O Stream?

A **Stream** is a sequence of data flowing from a source to a destination.

- **Input Stream:** Reads data from a source.
- **Output Stream:** Writes data to a destination.

Types of Streams: Byte vs. Character

Java categorizes streams based on the type of data they handle:

Feature	Byte Stream	Character Stream
Data Unit	8-bit bytes.	16-bit Unicode characters.
Use Case	Images, videos, executable files.	Text files (handles international languages).
Main Classes	InputStream, OutputStream.	Reader, Writer.

47) What is the File Class?

The File class is an abstract representation of file and directory pathnames. It **does not** read or write the actual content of the file; instead, it is used to:

- Check if a file exists.
- Create new files or directories.
- Delete files.
- Check file properties (size, read/write permissions).

48) what is Serialization and Deserialization

- **Serialization:** The process of converting an object's state (its variables) into a **byte stream** so it can be saved to a file or sent over a network. To make a class serializable, it must implement the `java.io.Serializable` interface.

- **Deserialization:** The reverse process—converting a byte stream back into a live Java object in memory.

49) What is the transient Keyword?

The transient keyword is used in serialization. If you mark a variable as transient, it will **not** be serialized.

- **Use Case:** Security (e.g., don't save a user's password to a file) or saving space (e.g., don't save a calculated value that can be recomputed).

50) FileInputStream vs. BufferedInputStream

- **FileInputStream:** Reads data directly from a file byte-by-byte. This is "expensive" because every read request triggers a call to the Operating System.
- **BufferedInputStream:** It creates an internal **buffer** (a temporary memory block). It reads a large chunk of data from the file into the buffer at once. Subsequent read calls pull from memory, making it **significantly faster**.

51) What is NIO (New I/O)?

Introduced in Java 1.4, **NIO** is an alternative to standard I/O.

- **Standard I/O:** Is **blocking** (the thread waits until data is read) and **stream-oriented**.
- **NIO:** Is **non-blocking** and **buffer-oriented**. It uses "Channels" and "Selectors," making it much more scalable for high-performance applications (like web servers).

52) What is a Thread?

A **Thread** is the smallest unit of execution within a process. It is often called a "lightweight process." Every Java program has at least one thread: the **main thread**.

53) Difference between Process and Thread

Feature	Process	Thread
Definition	An executing instance of a program.	A subset of a process.

Feature	Process	Thread
Memory	Has its own memory space.	Shares memory with other threads in the same process.
Overhead	Heavyweight (expensive to create/stop).	Lightweight (cheap to create/stop).
Communication	Difficult (Inter-Process Communication).	Easy (shares same objects/variables).

54) How to Create a Thread

There are two primary ways:

1. **Extending the Thread class:** Override the run() method.
2. **Implementing the Runnable interface:** Pass an instance of your class to a Thread object.

Runnable vs. Thread Class?

- **Implementing Runnable is preferred.** Since Java doesn't support multiple inheritance, extending the Thread class prevents you from extending any other class. Implementing the interface keeps your class flexible.

55) Thread Lifecycle

A thread can be in one of the following states:

1. **New:** Thread is created but start() hasn't been called.
2. **Runnable:** start() is called; the thread is ready to run and waiting for CPU time.
3. **Running:** The thread is currently executing its run() method.
4. **Blocked/Waiting:** The thread is alive but waiting for a resource or another thread (e.g., waiting for a lock).
5. **Timed Waiting:** Waiting for a specific period (e.g., Thread.sleep(1000)).
6. **Terminated (Dead):** The run() method has finished execution.

56) Synchronization, Deadlock, and Race Conditions

- **Synchronization:** A mechanism that ensures only one thread can access a resource (like a method or block of code) at a time. This is achieved using the `synchronized` keyword.
- **Race Condition:** Occurs when two or more threads try to modify the same data simultaneously, leading to unpredictable results.
- **Deadlock:** A situation where Thread A is waiting for a resource held by Thread B, and Thread B is waiting for a resource held by Thread A. They both wait forever.

57) What is the `volatile` Keyword?

The `volatile` keyword ensures that a variable is always read from and written to **main memory**, rather than being cached in a thread's local CPU cache. It guarantees "visibility"—if one thread changes the value, all other threads see the change immediately.

58) Executor Framework and Thread Pools

In professional Java development, we rarely create threads manually using `new Thread()`. Instead, we use the **Executor Framework**.

- **Thread Pool:** A collection of pre-created, "worker" threads. Instead of creating and destroying threads constantly (which is expensive), we submit tasks to the pool, and the pool assigns them to available threads.
- **Executor Framework:** A high-level API for managing thread pools.
 - *Example:* `ExecutorService executor = Executors.newFixedThreadPool(10);`

59) what are Concurrency Utilities

Introduced in `java.util.concurrent`, these are advanced tools for handling complex threading scenarios:

- **Concurrent Collections:** Thread-safe versions of collections (e.g., `ConcurrentHashMap`).
- **Atomic Variables:** Variables like `AtomicInteger` that allow thread-safe operations without using `synchronized`.
- **Locks:** More flexible than `synchronized` (e.g., `ReentrantLock`).

- **CountDownLatch/CyclicBarrier:** Tools to synchronize the starting or ending points of multiple threads.

60) Memory Management and String Handling

Java handles memory automatically, which is one of its greatest strengths. However, understanding how it works under the hood is crucial for writing efficient code.

Heap vs. Stack Memory

In the JVM, memory is divided into two main parts:

Feature	Stack Memory	Heap Memory
Purpose	Stores local variables and function calls.	Stores all Objects and their instance variables.
Size	Small and fixed.	Large and dynamic.
Access	LIFO (Last-In-First-Out) — very fast.	Random access — slower than stack.
Lifecycle	Exists only as long as the method is running.	Exists until the object is no longer referenced.
Failure	Leads to StackOverflowError.	Leads to OutOfMemoryError.

61) Garbage Collection (GC)

Garbage Collection is the automatic process of identifying and deleting objects that are no longer reachable or needed by the program. This frees up heap memory.

- **How it works:** The GC looks for objects that have no "references" pointing to them. If an object is unreachable, it is eligible for collection.
- **System.gc():** You can *request* GC to run using System.gc(), but the JVM is not forced to follow your request.

Types of Garbage Collectors

1. **Serial GC:** Uses a single thread for GC. Good for small applications.
2. **Parallel GC:** Uses multiple threads for GC. Known as the "throughput collector."
3. **G1 (Garbage First) GC:** Designed for large heaps; it splits the heap into regions to minimize "stop-the-world" pauses.
4. **ZGC:** A low-latency garbage collector designed for massive heaps (terabytes).

62)what is Immutable Class

An Immutable Class is a class whose state (data) cannot be changed after it is created.

- How to create one: 1. Declare the class as final (so it can't be extended). 2. Make all fields private and final. 3. Do not provide setter methods. 4. Initialize all fields via a constructor.

Why is String Immutable?

1. **Security:** Parameters like usernames, passwords, or URLs are often passed as strings. If strings were changeable, a hacker could change the value after a security check.
2. **String Pool:** If strings weren't immutable, changing one reference would accidentally change the value for all other variables pointing to that same pool object.
3. **Thread Safety:** Since they can't change, they are inherently thread-safe without needing synchronization.
4. **Caching:** Their hashCode is cached at creation, making them very fast when used as keys in a HashMap.

63) what is String Pool

The **String Constant Pool** is a special memory area inside the Heap.

- **Purpose:** To save memory by sharing String objects.
- **Mechanism:** When you create a string like `String s = "Hello";`, Java checks the pool first. If "Hello" already exists, it gives you a reference to the existing object instead of creating a new one.

64) String vs. StringBuilder vs. StringBuffer

If you are doing a lot of string manipulation (like a loop adding characters), you should avoid the String class because it creates a new object every time.

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable (cannot change).	Mutable (can change).	Mutable (can change).
Thread-Safe	Yes (inherently).	No (not synchronized).	Yes (synchronized).

Feature	String	StringBuilder	StringBuffer
Performance	Slow for frequent updates.	Fastest.	Slow (due to overhead).
Usage	Use for constant values.	Use for single-threaded logic.	Use for multi-threaded logic.

JDBC(Java Database Connectivity)

65) What is JDBC and components of JDBC?

JDBC (Java Database Connectivity) is a part of the Java Standard Edition (Java SE) that provides a set of classes and interfaces to execute SQL statements. Before JDBC, developers had to use platform-specific APIs; JDBC allows you to write database code once and run it against different databases using specific Drivers.

Main Components of JDBC

The JDBC architecture consists of four main components that work together:

1. JDBC API

This is a collection of interfaces and classes provided by Java (java.sql and javax.sql packages). Common ones include:

- **Connection:** Manages the communication with the database.
- **Statement / PreparedStatement:** Used to submit SQL queries.
- **ResultSet:** Holds the data retrieved from a database after a SELECT query.

2. JDBC Driver Manager

This is the backbone of the JDBC architecture. It is a class that manages a list of database drivers. It matches connection requests from the Java application with the proper database driver using a **Connection URL** (e.g., jdbc:mysql://localhost:3306/mydb).

3. JDBC Drivers

A JDBC Driver is a software component that enables a Java application to interact with a specific database. Think of it as a "translator." Each database (MySQL, Oracle, etc.) requires its own driver.

- **Type 4 Driver (Native-Protocol All-Java):** The most commonly used driver today. It converts JDBC calls directly into the network protocol used by the DBMS.

4. JDBC Test Suite (and the Data Source)

While the "Data Source" is technically the database itself (like MySQL), the **DataSource interface** is a modern component used in enterprise applications to manage connection pooling, providing a more efficient way to get connections than the DriverManager.

66) What is the Purpose of the JDBC API?

The primary goal of JDBC is to provide a uniform way to access data stored in relational databases. Specifically, its purposes include:

1. **Connecting to a data source:** Establishing a session between the Java app and the database.
2. **Sending Queries and Update Statements:** Passing SQL commands (like SELECT, INSERT, UPDATE, DELETE) to the database.
3. **Processing Results:** Retrieving the data returned by the database and mapping it to Java objects or variables.
4. **Transaction Management:** Handling commits and rollbacks to ensure data integrity.

67) JDBC Workflow

1. **Load the Driver** (usually done automatically in modern JDBC).
2. **Establish Connection** via DriverManager.getConnection().
3. **Create a Statement** object.
4. **Execute the Query** and get a ResultSet.
5. **Process the Result** using a loop.
6. **Close the Connection** to free up resources.

68) What are JDBC Drivers?

A **JDBC Driver** is a software component that enables a Java application to interact with a database. It implements the interfaces defined in the java.sql package (like Connection, Statement, and ResultSet) specifically for a particular database engine (e.g., MySQL, Oracle, PostgreSQL).

Types of JDBC Drivers

There are four distinct types of drivers, categorized by how they bridge the gap between Java and the database:

Type	Name	How it Works	Pros/Cons
Type 1	JDBC-ODBC Bridge	Translates JDBC calls into ODBC (Open Database Connectivity) calls.	Deprecated. Requires ODBC installation on every client machine. Slow.
Type 2	Native-API Driver	Translates JDBC calls into native C/C++ client-side API calls of the database.	Faster than Type 1 but requires native database client libraries installed on every machine.
Type 3	Network-Protocol Driver	Translates JDBC calls into an intermediate server-side protocol, which then talks to the database.	Fully Java-based and doesn't need client-side libraries, but requires an extra server layer.
Type 4	Thin Driver (Native-Protocol)	Converts JDBC calls directly into the network protocol used by the DBMS.	Most common. Written entirely in Java. No client-side installation or middleware needed.

Which JDBC driver is most commonly used and why?

The **Type 4 Driver (Thin Driver)** is the industry standard and the most widely used.

Why?

- **Pure Java:** It is written entirely in Java, making it platform-independent.
- **No Installation:** You don't need to install native database client software (like Oracle Client) on the user's machine; you just include the .jar file in your project.
- **Performance:** Since it communicates directly with the database server over the network without intermediate layers (like ODBC or native APIs), it is highly efficient.
- **Ease of Deployment:** You simply drop the driver into your classpath, and you're ready to go.

69) What is the role of DriverManager?

The DriverManager class acts as an **interface between the user and the drivers**. Its primary responsibilities include:

1. **Registering Drivers:** It keeps track of all the JDBC drivers available. In modern Java (JDBC 4.0+), drivers found in the classpath are registered automatically.
2. **Establishing Connections:** When you call `DriverManager.getConnection(url, user, password)`, the `DriverManager` iterates through its list of registered drivers.
3. **Matching the URL:** It asks each driver, "Can you handle this URL (e.g., `jdbc:mysql://...`)?" The first driver that recognizes the sub-protocol (the `mysql` part) creates and returns the `Connection` object.
4. **Logging:** It can handle basic logging and tracing of JDBC operations for debugging.

70) Steps to connect to database using JDBC

Database Connection

- **Steps to connect:** 1. **Import packages** (`java.sql.*`). 2. **Load/Register Driver** (Automatic in JDBC 4.0+). 3. **Establish Connection** using `DriverManager.getConnection()`. 4. **Create Statement** object. 5. **Execute Query** and get results. 6. **Close Connections** to free resources.
- **How to establish:** Use `Connection conn = DriverManager.getConnection(url, user, password);`.
- **What is Connection Object:** A session between the Java application and the database. It handles transactions and creates statement objects.

71) Statements in JDBC

- **Statement:** Used for general-purpose access. Best for static SQL (no parameters).
- **PreparedStatement:** Pre-compiled SQL. Best for dynamic queries with parameters (?).
- **CallableStatement:** Used to execute Stored Procedures in the database.
- **Difference:** Statement is compiled every time; PreparedStatement is compiled once and reused.
- **Why Preferred?** It is faster (due to pre-compilation) and prevents SQL Injection.

72) SQL Execution

What is `executeQuery()`, `executeUpdate()`, `execute()`

Method	Purpose	Return Type
executeQuery()	SELECT statements.	ResultSet
executeUpdate()	INSERT, UPDATE, DELETE, CREATE.	int (rows affected)
execute()	Any SQL. Use if the query type is unknown.	boolean (true if it returns a ResultSet)

73) ResultSet

- **What is it?** A table of data representing a database result set.
- **How to retrieve:** Use a loop with `.next()` and getter methods like `getString("column_name")`.
- **Types:**
 - * **TYPE_FORWARD_ONLY:** Cursor moves only forward (Default).
 - **TYPE_SCROLL_INSENSITIVE:** Cursor can move back and forth; ignores database changes.
 - **TYPE_SCROLL_SENSITIVE:** Cursor moves both ways; reflects database changes.
- **Cursor:** A control structure that points to the current row of data in the ResultSet.

74) Transactions

- **Transaction:** A sequence of operations treated as a single unit (All or Nothing).
- **Auto-commit mode:** By default, JDBC commits every SQL statement immediately.
- **Commit/Rollback:** To manage manually, set `conn.setAutoCommit(false)`. Use `conn.commit()` to save or `conn.rollback()` to undo if an error occurs.
- **Management:** Ensuring the **ACID** properties (Atomicity, Consistency, Isolation, Durability) of data.

75) Exception Handling & Security

- **SQLException:** The primary exception class for database errors (e.g., wrong syntax, connection timeout).
- **SQL Injection:** A hack where an attacker inserts malicious SQL into a query via input fields (e.g., ' OR '1'='1').
- **Prevention:** PreparedStatement treats inputs as **data values**, not executable code, making injection impossible.

76) Performance & Advanced

- **Batch Processing:** Grouping multiple SQL statements together and sending them to the database in one "trip" to reduce network overhead.
- **Connection Pooling:** A cache of database connections. Instead of opening/closing a connection for every user (which is slow), the app "borrows" a ready-made one from the pool (e.g., HikariCP).
- **RowSet:** A wrapper around a ResultSet that is easier to use and can be "disconnected" from the database.

77) How do you close JDBC resources

- **Closing Resources:** Always close ResultSet, Statement, and Connection in a finally block or use **Try-with-Resources** (Java 7+) to prevent memory leaks.
- **Best Practices:**
 1. Use **Connection Pooling**.
 2. Always use **PreparedStatement**.
 3. Use **Try-with-Resources**.
 4. Don't use SELECT *; fetch only the columns you need.
 5. Handle **SQLExceptions** specifically.

Java

// Modern Best Practice Example

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
```

```
    PreparedStatement pstmt = conn.prepareStatement("SELECT name FROM users WHERE  
id = ?")) {
```

```
pstmt.setInt(1, 101);  
try (ResultSet rs = pstmt.executeQuery()) {  
    while (rs.next()) {  
        System.out.println(rs.getString("name"));  
    }  
}  
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

